

Les bases de données relationnelles

Introduction aux commandes DDL en SQL

DML

Introduction

- DML (Data Manipulation Language) permet de manipuler les données à l'intérieur des tables.
- Il comprend les commandes :
 - SELECT : Récupérer des données.
 - INSERT : Ajouter de nouvelles données.
 - UPDATE : Modifier des données existantes.
 - DELETE : Supprimer des données.
- Les commandes DML sont généralement contrôlées par des transactions (TCL) et peuvent être annulées avec ROLLBACK.

DML

SELECT - Récupération de données

- Utilité : Extraire des informations spécifiques depuis une ou plusieurs tables.
- Syntaxe de base :

```
SELECT colonne1, colonne2 FROM nom_table WHERE condition;
```

- Exemple :

```
SELECT nom, prix FROM articles WHERE prix > 50;
```

DML

SELECT - Alias

- Utilité : Donner un nom temporaire à une table ou une colonne dans une requête.
- Syntaxe :
- Pour une colonne :

```
SELECT colonne AS alias FROM table;
```

- Pour une table :

```
SELECT colonne FROM table AS alias; ou plus simplement SELECT colonne FROM alias table;
```

- Les alias rendent les requêtes plus lisibles, surtout lorsqu'il y a des jointures ou des sous-requêtes.
- Exemples :

```
SELECT nom AS nom_article, prix AS prix_article FROM Article;
```

```
SELECT c.nom, c.email FROM Client AS c WHERE c.id > 10;
```

```
SELECT c.nom, c.email FROM Client c WHERE c.id > 10;
```

DML

SELECT - Clauses importantes

- WHERE : Filtrer les résultats.
- ORDER BY : Trier les résultats.
- GROUP BY : Grouper les résultats.
- LIMIT : Limiter le nombre de résultats.

DML

SELECT - Jointures (JOIN)

- Utilité : Combiner des données de plusieurs tables liées par des clés étrangères.
- Types de jointures :
 - INNER JOIN : Retourne les correspondances communes.
 - LEFT JOIN : Retourne tous les éléments de la table de gauche.
 - RIGHT JOIN : Retourne tous les éléments de la table de droite.
- Exemple :

```
SELECT clients.nom, commandes.date FROM clients  
INNER JOIN commandes ON clients.id = commandes.client_id;
```

DML

INSERT - Ajout de données

- Utilité : Ajouter de nouvelles lignes dans une table.

- Syntaxe :

```
INSERT INTO nom_table (colonne1, colonne2) VALUES (valeur1, valeur2);
```

- Exemple :

```
INSERT INTO articles (nom, prix, stock) VALUES ('Nouveau Produit', 75.00, 10);
```

DML

UPDATE - Modification de données

- Utilité : Mettre à jour des données existantes dans une table.
- Syntaxe :

```
UPDATE nom_table SET colonne1 = nouvelle_valeur WHERE condition;
```

- Exemple :

```
UPDATE articles SET prix = 80.00 WHERE nom = 'Nouveau Produit';
```

DML

DELETE - Suppression de données

- Utilité : Supprimer des lignes d'une table.
- Syntaxe :

```
DELETE FROM nom_table WHERE condition;
```

- Exemple :

```
DELETE FROM articles WHERE stock = 0;
```

DML

Bonnes pratiques

- Utiliser des transactions pour garantir l'intégrité des données (Nous aborderons ceci dans la partie TCL).
- Filtrer avec soin les opérations UPDATE et DELETE pour éviter les erreurs.
- Optimiser les requêtes SELECT pour des performances optimales (index, etc.).

DML

Exercice pratique

1. Insérer trois nouveaux articles dans la table Article.
 1. Nom : "Clavier mécanique Gamer"
 - Prix : 129.99
 - Stock : 25
 2. Nom : "Souris Ergonomique sans Fil"
 - Prix : 49.99
 - Stock : 50
 3. Nom : "Écran Ultrawide 34 pouces"
 - Prix : 349.99
 - Stock : 10
2. Sélectionner tous les articles dont le prix est compris entre 50€ et 100€.
3. Augmenter le prix de tous les articles de 5€ pour les articles dont le stock est supérieur à 2.
4. Supprimer les articles dont le stock est inférieur à 2.